

Devoir surveillé d'informatique

 Consignes

- La calculatrice n'est **pas autorisée**.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veuillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Tri par sélection

Q1– Expliquer brièvement le principe de l'algorithme du *tri par sélection* et donner l'évolution du contenu de la liste [42, 28, 11, 9, 33] lorsqu'elle est triée en utilisant cet algorithme.

On parcourt la liste avec un indice i à partir du début. Pour chaque valeur de i on sélectionne l'indice m du minimum des éléments de la liste à partir de l'indice i et on l'échange avec celui d'indice i . Sur la liste [42, 28, 11, 9, 33], on obtient successivement :

- [9, 28, 11, 42, 33]
- [9, 11, 28, 42, 33]
- [9, 11, 28, 42, 33]
- [9, 11, 28, 33, 42]

Q2– Ecrire en Python une fonction `echange` qui prend en argument une liste `lst` ainsi que deux entiers i et j ne renvoie rien et échange les éléments d'indice i et j de cette liste. Par exemple, si `lst = [25, 17, 34, 22, 37]` alors `echange(lst, 1, 3)` doit modifier la liste `lst` en [25, 22, 34, 17, 37]. On vérifiera les préconditions sur i et j à l'aide d'instructions `assert`.

```
1 def echange(lst, i, j):
2     '''Echange les éléments d'indice i et j dans lst'''
3     assert 0 <= i < len(lst) and 0 <= j < len(lst)
4     lst[i], lst[j] = lst[j], lst[i]
```

Q3– Ecrire en Python une fonction `indice_min_depuis` qui prend en argument une liste `lst` et un entier i et renvoie l'indice du plus petit élément de la liste `lst` à partir de l'indice i . Par exemple, si `lst = [25, 17, 34, 22, 37]` alors `indice_min_depuis(lst, 2)` doit renvoyer 3 (c'est à dire l'indice de 22 qui est l'élément minimal à partir de celui d'indice 2).

```
1 def indice_min_depuis(lst, i):
2     '''Renvoie l'indice du minimum de lst à partir de l'indice i'''
3     imin = i
4     for j in range(i+1, len(lst)):
5         if lst[j] < lst[imin]:
6             imin = j
7     return imin
```

Q4– Ecrire en Python une fonction `tri_selection` qui prend en argument une liste `lst` et modifie cette liste de façon à la trier par ordre croissant en utilisant l'algorithme du tri par sélection. On utilisera les fonctions `echange` et `indice_min_depuis` écrites aux questions précédentes.

```

1 def tri_selection(liste):
2     '''Trie la liste par sélection'''
3     for i in range(len(liste)):
4         imin = indice_min_depuis(liste, i)
5         echange(liste, i, imin)

```

- Q5–** Un ordinateur a pris 3 secondes pour trier une liste de 250 000 éléments en utilisant un tri par sélection. Donner une estimation (aucune justification n'est demandée) du temps que prendrait cet ordinateur pour trier une liste de 1 000 000 éléments en utilisant le même algorithme.

On peut estimer que le temps de calcul est proportionnel au carré du nombre d'éléments à trier. Ainsi, si le temps est de 3 secondes pour 250 000 éléments, il serait de $3 \times \left(\frac{1\,000\,000}{250\,000}\right)^2 = 3 \times 16 = 48$ secondes pour 1 000 000 éléments. La justification n'était pas demandée mais en examinant l'algorithme on peut constater que le nombre d'opérations à effectuer évolue comme le carré du nombre d'éléments à trier.

□ **Exercice 2 : Écriture binaire des entiers**

- Q6–** Donner l'écriture en base 2 de $(172)_{10}$.

$$(172)_{10} = (10101100)_2$$

- Q7–** Ecrire $(10011101)_2$ en base 10.

$$\text{C'est } (10011101)_2 = 1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 128 + 16 + 8 + 4 + 1 = 157$$

- Q8–** Quel est le plus grand nombre représentable en base 2 en utilisant 10 bits ?

$$\text{Le plus grand nombre représentable en base 2 avec 10 bits est } (1111111111)_2 = 2^{10} - 1 = 1023$$

- Q9–** On suppose que l'on travaille sur des entiers codés sur 8 bits en complément à 2. A quel nombre entier en base 10 correspond $(11011001)_2$?

$$\text{Le bit de poids fort est compté négativement et les autres positivement. Donc } (11011001)_2 = -2^7 + 2^6 + 2^4 + 2^3 + 2^0 = -128 + 64 + 16 + 8 + 1 = -39$$

□ **Exercice 3 : Exponentiation**

Dans cet exercice, on s'intéresse à différentes fonctions permettant de calculer pour deux entiers positifs $a \in \mathbb{N}$ et $n \in \mathbb{N}$, la valeur de a^n . On s'interdit l'utilisation de l'opérateur d'exponentiation ****** de Python qui donnerait directement le résultat.

- Q10–** Ecrire une fonction *iterative* **expo_iter** qui prend en argument deux entiers **a** et **n** et renvoie **a** puissance **n** en procédant par multiplication successive par **a**.

```

1 def expo_iter(a, n):
2     p = 1
3     for i in range(n):
4         p = p * a
5     return p

```

- Q11–** Exprimer, pour $n > 0$, a^n en fonction de a^{n-1} puis écrire une version *réursive* **expo_rec** de la fonction permettant de calculer a^n .

On a $a^n = a \times a^{n-1}$ et $a^0 = 1$, ces relations permettent d'écrire une version récursive de la fonction d'exponentiation.

```

1 def exp_rec(a, n):
2     if n==0:
3         return 1
4     return a * expo_rec(a, n-1)

```

On donne ci-dessous un algorithme appelé *exponentiation rapide* :

Algorithme : Exponentiation rapide

Entrées : $a \in \mathbb{N}, n \in \mathbb{N}$

Sorties : a^n

```

1 p ← 1
2 tant que n ≠ 0 faire
3     si n est impair alors
4         p ← p × a
5     fin
6     a ← a * a
7     n ← ⌊ n/2 ⌋
8 fin
9 return p

```

- Q12**– Donner les valeurs successives prises par les variables a , n et p si on fait fonctionner cet algorithme avec $a = 2$ et $n = 13$. On pourra recopier et compléter le tableau suivant et donner les valeurs de a et de p sous la forme de puissance de 2 :

	a	n	p
valeurs initiales	2	13	1
après un tour de boucle	2^2	6	2
après deux tours de boucle	2^4	3	2
après trois tours de boucle	2^8	1	2^5
après quatre tours de boucle	2^{16}	0	2^{13}

- Q13**– Ecrire une implémentation en Python de cet algorithme sous la forme d'une fonction `exp_rapide`.

```

1 def exp_rapide(a, n):
2     p = 1
3     while n!=0:
4         if n%2==1:
5             p = p * a
6             a = a * a
7             n = n//2
8     return p

```

- Q14**– Prouver que cet algorithme termine, on pourra montrer que n est un variant de la boucle tant que de l'algorithme.

Dans l'algorithme ci-dessus, la quantité n est un variant de boucle, en effet :

1. $n \in \mathbb{N}$ par précondition.
2. n reste positif par condition d'entrée dans la boucle.
3. n décroît strictement car n est divisé par 2 lors de chaque passage dans la boucle.

L'algorithme termine car on a trouvé un variant de boucle.

- Q15**– Prouver que cet algorithme est correct. En notant a_0 (resp. n_0) la valeur initiale de a (resp. n), on pourra prouver l'invariant $p \times a^n = a_0^{n_0}$.

On note, a_0 la valeur initiale de a et n_0 la valeur initiale de n , montrons que la propriété I : « $p \times a^n = a_0^{n_0}$ » est un invariant de boucle.

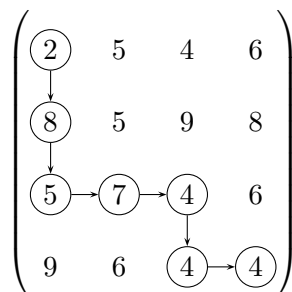
1. Avant d'entrée dans la boucle $p = 1$, $a = a_0$ et $n = n_0$ donc $p \times a^n = a_0^{n_0}$ et I est vérifiée.
2. On suppose I vérifié à l'entrée de la boucle et on note a' (resp. n' , resp. p') les valeurs prises par a (resp. n , resp. p) au tour de boucle suivant, alors :

- Si n est paire, $n' = n/2$, $p' = p$ et $a' = a^2$ donc
 $p' \times a'^{n'} = p \times (a^2)^{n/2}$
 $p' \times a'^{n'} = p \times a^n$
et puisque I était vraie en entrée de boucle, $p' \times a'^{n'} = a_0^{n_0}$
- Sinon, $n' = (n-1)/2$, $p' = p \times a$ et $a' = a^2$ donc
 $p' \times a'^{n'} = p \times a \times (a^2)^{(n-1)/2}$
 $p' \times a'^{n'} = p \times a^n$
et puisque I était vraie en entrée de boucle, $p' \times a'^{n'} = a_0^{n_0}$

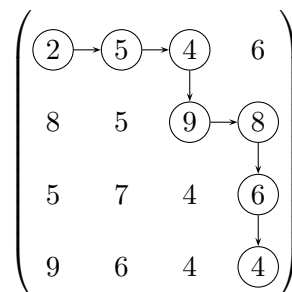
En sortie de boucle, puisque $n = 0$, cet invariant donne $p \times a^0 = a_0^{n_0}$ et donc $p = a_0^{n_0}$ et donc l'algorithme est correcte.

□ Exercice 4 : Traversée maximale d'une matrice carrée

On considère une matrice à n lignes et n colonnes notée M à coefficients dans $\mathbb{N}$, et dont le coefficient situé à la ligne i et à la colonne j avec $0 \leq i \leq n-1$ et $0 \leq j \leq n-1$ sera noté $M_{i,j}$. On s'intéresse aux chemins depuis la première valeur en haut à gauche ($M_{0,0}$) jusqu'à la dernière en bas à droite ($M_{n-1,n-1}$) qui n'utilisent que les déplacements vers la droite ($\rightarrow$) ou vers le bas ($\downarrow$). Voici deux exemples de tels chemins dans une même matrice A de 4 lignes et 4 colonnes :



Un exemple C_1 de chemin dans A



Un exemple C_2 de chemin dans A

On appelle *somme d'un chemin* dans une matrice M , la somme des coefficients $M_{i,j}$ rencontrés en suivant ce chemin. Dans les exemples ci-dessus, le chemin C_1 a une somme de 34 et le chemin C_2 a une somme de 38. On décide de représenter un chemin par une liste de 0 (un pas vers le bas) et de 1 (un pas à droite). Par exemple le chemin C_1 donné en exemple ci-dessus correspond à la liste $[0, 0, 1, 1, 0, 1]$. La matrice est représentée par une liste de liste d'entiers, par exemple la matrice A sera représentée par $[[2, 5, 4, 6], [8, 5, 9, 8], [5, 7, 4, 6], [9, 6, 4, 4]]$.

Q16– Montrer que pour une matrice de taille n , un chemin valide depuis $M_{0,0}$ jusqu'à $M_{n-1,n-1}$ contient $n-1$ fois le chiffre 0 et $n-1$ fois le chiffre 1.

On doit se déplacer $n-1$ fois vers la droite car on passe de la colonne 0 à la colonne $n-1$, le seul mouvement qui augmente la colonne étant le mouvement vers la droite, ce mouvement doit apparaître $n-1$ fois et donc il y a $n-1$ fois le chiffre 1 dans un liste de mouvement permettant de passer de $M_{0,0}$ à $M_{n-1,n-1}$. Par un raisonnement similaire sur les lignes, le chiffre 0 doit apparaître $n-1$ fois aussi.

Q17– Ecrire une fonction `est_valide` qui prend en argument un chemin et un entier n et renvoie `True` si et seulement ce chemin est un chemin valide depuis $M_{0,0}$ jusqu'à $M_{n-1,n-1}$.

```

1  def est_valide(chemin, n):
2      '''Teste si un chemin est valide :
3      il ne doit contenir que des 0 et des 1
4      il doit y avoir n-1 fois 0 et n-1 fois 1
5      '''
6      nb0 = 0
7      nb1 = 0
8      for pas in chemin:
9          if pas==0:
10             nb0 = nb0 + 1
11          elif pas==1:
12             nb1 = nb1 + 1
13          else:
14             return False
15      if (nb0==n-1 and nb1==n-1):
16          return True
17      else:
18          return False

```

Q18– Ecrire une fonction `somme_chemin` qui prend en argument une matrice et un chemin (supposé valide) et renvoie la somme obtenue en parcourant la matrice en suivant ce chemin.

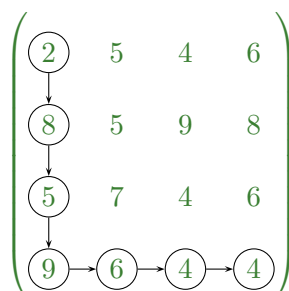
```

1  def somme_chemin(chemin, matrice):
2      lig = 0
3      col = 0
4      somme = matrice[lig][col]
5      for pas in chemin:
6          if pas==0:
7              lig = lig + 1
8          if pas==1:
9              col = col + 1
10         somme = somme + matrice[lig][col]
11     return somme

```

On propose l'algorithme suivant afin de résoudre ce problème : si les deux directions bas ou droite sont possibles on choisit celle qui mène vers le coefficient de matrice le plus élevé, en cas d'égalité, on choisit (arbitrairement) la direction bas. Dans l'exemple ci-dessus à partir de $M_{0,0} = 2$ on peut aller vers à droite vers $M_{0,1} = 5$ ou en bas vers $M_{1,0} = 8$ et on choisit donc d'aller en bas puisque $8 > 5$. Une fois arrivé en $M_{1,0}$ les deux directions possibles mènent vers des coefficients identiques (5 des deux côtés) et donc, on va vers le bas. Enfin, lorsqu'une seule direction est possible (parce qu'on a atteint la dernière colonne ou la dernière ligne), c'est cette direction qui est choisie.

Q19– Poursuivre le déroulement de cet algorithme sur la matrice A donnée en exemple ci-dessus, dessiner le chemin obtenu et donner sa somme.



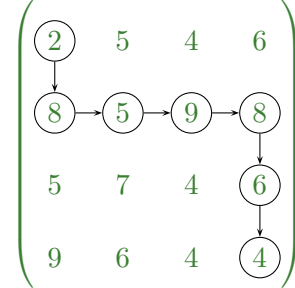
Le chemin obtenu a une somme de $2 + 8 + 5 + 9 + 6 + 4 + 4 = 38$

Q20– Dans quelle famille d'algorithme peut-on classer cet algorithme ? Justifier.

C'est un algorithme glouton car à chaque étape on choisit un maximum *local* sans se préoccuper de l'impact de ce choix sur le maximum global.

Q21– Montrer en exhibant un exemple dans le cas $n = 4$ que cet algorithme ne fournit pas toujours un chemin de somme maximale.

On a obtenu sur l'exemple une somme de 38, or dans cette même matrice, un autre chemin donne une somme supérieure :



Le chemin obtenu a une somme de $2 + 8 + 5 + 9 + 8 + 2 + 4 = 42$

Q22– Ecrire une fonction `greedy` qui prend en argument une matrice (représentée par une liste de liste) et renvoie le chemin (sous la forme d'une liste de 0 et de 1) obtenu en appliquant cet algorithme.

```

1  def greedy(matrice):
2      lig = 0
3      col = 0
4      somme = matrice[lig][col]
5      n = len(matrice)
6      while (lig < n-1 or col < n-1):
7          if lig == n-1:
8              col = col + 1
9          elif col == n-1:
10             lig = lig + 1
11             elif matrice[lig+1][col] >= matrice[lig][col+1]:
12                 lig = lig + 1
13             else:
14                 col = col + 1
15             somme = somme + matrice[lig][col]
16         return somme

```